#1 Check that the entry isn’t already in the cache (we get a different kind of object here).
#2 Instead of just moving the node to the front of the list, here we have to increment the counter by one, update the time of last access, and trickle the entry down towards the end of the queue.
#3 Add a new entry to the priority queue. Here the counter is 1, being a new entry.

The solution is to use a tuple, <counter, timestamp> as priority, where the timestamp is the time of last access to an entry and it’s used to break ties on the counter; here higher frequency and larger timestamps mean higher priority.

Finally, we have the method to evict one of the entries: using a priority queue makes its implementation even easier, because all the details of removing the element and updating the queue are encapsulated in the priority queue itself.

Listing 7.7 LFU Cache evictOneEntry (private method)

```
function LFUCache::evictOneEntry()
  if hashTable.isEmpty()
then
    return
false
  node ← elements.pop()
                            #1

hashTable.delete(node.getKey())
```